

Single-Decision Problems

ראינו בעיות תכנון שבהן מה שמעניין אותנו זה המסלול ממצב התחלתי למצב מטרה.

- במקרה הדטרמיניסטי
 - במקרה הסטוכסטי (שם במקום תוכנית יש לנו policy)
 - במקרה בו ההסתברויות לא ידועות (וצריך לעשות reinforcement learning)
- עכשיו נסתכל על בעיה של מציאת פתרון שבה המסלול לא חשוב לנו, אלא רק למצוא את מצב המטרה. יש לנו פונקציה שיודעת להשוות בין שני מצבים - ואנחנו צריכים למצוא את הפתרון הכי טוב.
- בעצם:
- בבעיות תכנון אנחנו מחפשים את המסלול האופטימלי (ולפעמים מסתפקים בכמעט-אופטימלי)
 - בבעיות single decision אנחנו מחפשים את הקודקוד האופטימלי
 - למשל: מציאת שיבוץ אופטימלי. לא מעניין אותנו סדר הפעולות שיוביל אותנו לשיבוץ האופטימלי - אלא רק השיבוץ הסופי עצמו.
 - סידור החיבורים על מעגלים אלקטרוניים.
 - סודקו
 - תשבצים
- כל הפרספקטיבה משתנה::
- עדיין יש גרף - שמייצג את מה שאנחנו יודעים על האילוצים - אבל אין משמעות להליכה על הגרף (המשקל של כל קשת הוא אפס)
 - לעומת זאת לצמתים עצמם יש פונקציית הערכה $B(s)$ ואנחנו מחפשים את state s עם ה- $B(s)$ הטוב ביותר.
 - בבעיות $B, \text{satisfaction}$ הוא פרדיקט - אומר אם s מתאים או לא.
 - בבעיות $B, \text{optimization}$ הוא ערך מספרי.
 - אין חשיבות לסדר הפעולות - רק לתוצאה הסופית.
- בעיות single decision יכולות להיות מורכבות מ:
- טענות בוליאניות (propositions)
 - משתנים מספריים. יכולים להיות integers או real.
 - קטגוריות (למשל בעיית צביעה של מפה)
 - אילוצים במרחב.

Constraint Satisfaction Problems

ייצוג: בעיית CSP מיוצגת ע"י שלושה דברים:

1. משתנים $V = \{V_1, \dots, V_N\}$ (כלומר $N = |V|$)
2. לכל משתנה יש $\text{DOMAIN}(V_i)$ שמגדיר איזה ערכים הוא יכול לקבל.
3. יש אילוצים על המשתנים.
 - אילוץ הוא פונקציה על משתנה אחד או יותר.
 - אילוצים על משתנה בודד נקראים "אונריים".
 - אילוצים על שני משתנים נקראים "בינאריים".

המטרה:

- כל המשתנים יקבלו ערך
- כל האילוצים יקוימו

דוגמאות: 1. k-sat

2. צביעת גרף

3. scheduling. יש תלויות בין המשימות - למשל $\text{Start}(J_1) + 5 \leq \text{Start}(J_3)$

כחלק מהאלגוריתמיקה, יהיה לנו נוח לייצג את המידע (או לפחות חלק ממנו) בצורת גרף עם שני סוגים של צמתים - צומת אילוץ (נקראים arcs) וצומת משתנה - והקשרים בין הצמתים יאפשרו לנו להפעיל יוריסטיקות.

נשים לב: צומת משתנה לא מחובר ישירות לצומת משתנה אחר, גם אם יש אילוץ שכולל את שניהם. למשל אם יש אילוץ שאומר שכל המשתנים שונים אחד מהשני, לא נראה full mesh בין כל המשתנים - אלא צומת אילוץ שמחובר לכל צמתי המשתנים.

הגרף תופס רק את המבנה - לא את התוכן.

גישות: 1. חיפוש סיסטמטי (informed backtracking)

מובטח שימצא פתרון - או יוכיח שאין פתרון. כלומר זהו אלגוריתם שהוא complete.

ברור שזה מתפוצץ - ולכן נצטרך יוריסטיקות.

2. חיפוש לוקאלי (iterative repair)

מתחילים ממצב אקראי, ומנסים לתקן/לשנות משתנים. אפשר גם לעשות restart אם רואים שזה לא מצליח.

לא מובטח לנו שזה יצליח.

צריך לדעת איך למדל נכון. לדוגמה - בעיית N המלכות. בלוח שחמט של $N \times N$, איך למקם N מלכות

ייצוגים: 1. לכל מלכה לייצג (c, r)

יש כאן N^{2N} מצבים.

2. לכל שורה (r) לייצג את הטור c בה עומדת המלכה.

יש רק N^N מצבים. זה עדיין אקספוננציאלי - אבל יותר קטן.

- פתרונות: חיפוש סיסטמטי - מציבים את המלכות אחת אחת, ואם רואים שיש מלכה שאי אפשר להציב חוזרים אחורה ומנסים את המלכה הקודמת במקום אחר.
- חיפוש לוקלי - מתחילים מהצבה אקראית (עם אילוץ בסיסי של מלכה בכל טור) ובכל איטרציה בוחרים מלכה אחת ומזיזים אותה למקום אחר באותו טור.

Backtracking

- עוברים משתנה משתנה
- לכל משתנה עוברים ערך ערך בdomain שלו
- כל search node הוא $\langle A, V, C \rangle$ כאשר:
 - A ASSIGNMENT (n) - סט ההשמות.
 - V VARS (n) - סט המשתנים ללא השמה.
 - C CONSTRAINTS (n) - אוסף האילוצים¹.
- האלגוריתם הפשוט:
 - מתחילים מ- $n = \langle \theta, V, C \rangle$
 - בכל פעם בוחרים משתנה v , ולכל ערך אפשרי שלו $h \in \text{DOMAIN}(v)$ מוסיפים את $A \cup \{v = h\}$.
 - בודקים $\text{CONSISTENT}(A \cup \{v = h\}, V \setminus \{v\}, C)$
 - אם כן - יורדים ברקורסיה² עם ההצבה הזאת.
- האלגוריתם הזה מתפוצץ מאוד מהר. הוא נתקע שוב ושוב באותן בעיות. נרצה לעשות משהו יותר יעיל, שישתמש במבנה האילוצים - לבחור את המשתנים בסדר שיאפשר לנו לפסול יותר מהר.
- בשביל זה נרצה יוריסטיקות:
 - SelectVar () - יוריסטיקה שבחרת את המשתנה הבא. בניגוד לplannig, בה היוריסטיקה אמורה לתת לנו משהו שאמור להצליח, כאן יוריסטיקה טובה היא כזאת שתוביל אותנו לפסילה כמה שיותר מהר.
 - שתי יוריסטיקות עיקריות לSelectVar:
 - minimum remainnig value - לבחור את המשתנה שמספר הערכים שהוא יכול לקבל הוא הנמוך ביותר.
 - זה טוב כי זה מקטין את branching, וכי זה מאפשר להגיע יותר מהר לערך לא חוקי.

¹בגירסה הפשוטה C לא משתנה, אבל אפשר לעשות שיפורים שבהם הוא כן משתנה.
²לא באמת לעשות רקורסיה - זה מאוד לא יעיל.

– most constrained variable - המשתנה שמשפיע על הכי הרבה אילוצים (דרגה הכי גבוהה בגרף)

אפשר לשלב - לעשות minimum remaining value ומשתמשים בmost constrained variable בתור tie breaker.

• OrderValues () - יוריסטיקה שמסדרת את הערכים של המשתנה בסדר שנרצה לבדוק אותם

כאן דווקא בוחרים לנסות קודם ערכים שינסו להצליח.

נשים לב: בניגוד לplanning, כאן אין משמעות לאדמיניסטרציה של יוריסטיקה.

Consistency Lookahead

נשמור את $L(v)$ - סט הערכים החוקיים של כל משתנה. מתחיל בתור $L(v) = \text{Domain}(v)$, ובכל פעם, לפי האילוצים, מורידים ממנו ערכים. אם הוא ריק - קוטמים. בניגוד ליוריסטיקה, כאן הקטימה דטרמיניסטית ושלמה. יש שתי פונקציות:

• UpdateAhead () - עוברת על כל המשתנים שעוד נותרו, לפי הגרף בודקת מי מהם קשור להשמה החדשה, ומורידה מ $L(v)$ את הערכים שכבר לא אפשריים.

• LookAhead () - מחזיר את הערכים שנשארו

אפשר להשתמש במבנה הנתונים הזה בשביל היוריסטיקה של minimum remaining value.

Constraint Propagation

ברגע שעושים אילוץ, אפשר לבדוק את ההשפעה שלו גם על אחרים. אבל זה מוגבל לזוגות (כי אחרת זה פשוט יעשה brute force על CSP עצמו)

חיפוש לוקאלי - iterative repair

דומה לgradient descent - רוצים בכל איטרציה למצוא מצב קרוב שישפר אותנו. בשביל לשפר, צריך לבחור משתנה ולבחור לו ערך אחר. אפשר לבחור את המשתנה באקראי - כל עוד זה משתנה שמפר אילוץ³, אבל בשביל הערך צריך יוריסטיקה. למשל - לבחור את הערך שמפר הכי פחות אילוצים (אם יש כמה - בוחרים אקראית). אם נתקעים, אפשר לעשות reset - לבחור השמה אקראית חדשה. עבור בעיית N המלכות זה עובד טוב מאוד ומתכנס מאוד מהר. גם עבור N ימים גדולים מאוד. מה שחשוב זה בעצם היחס:

$$R = \frac{\text{number of constraints}}{\text{number of variables}}$$

היינו מצפים שככל R יהיה יותר גדול ככה יהיה יותר קשה למצוא פתרון - עד לרגע שהוא מספיק גדול שקל להוכיח שאין פתרון. בפועל גילו שזה אמנם ככה - אבל לא הדרגתי, אלא

³לכאורה אפשר לשפר - למשל לבחור משתנה שנמצא שמפר הכי הרבה אילוצים - אבל בפועל גם באקראי זה עובד טוב.

בינה מלאכותית
89-5570-06

מקליד: עידן אריה
מרצה: פרופ' גל קמינקא
תאריך: 2026-06-18

רוב הגרף עם CPU time מאוד נמוך ויש אזור מאוד ספציפי של R שבו CPU timeen מאוד גבוה. האזור הזה מתאים לסיגמואיד (הפוך) שמתאר את ההסתברות שיש פתרון. האזור הזה נקרא phase transition, והוא תלוי בעיקר בשפה שמגדירה את הבעיה. הוא לא תלוי באלגוריתם - אבל כן בייצוג (שיכול לשנות את היחס)